

MY CYBER ACADEMY

Unit U12 of 14

CREST Practitioner Intrusion Analyst (CPIA)

Reverse Engineering Malware I: Intro, Anti-RE & Functionality

The goals of malware analysis, indicators of compromise, OPSEC, and the anti-reverse-engineering arms race.

GROUP F | MALWARE ANALYSIS | ANTI-RE

© Billy Forrest

Three sessions on reverse engineering malware

Session	Focus	Skill areas
12 · Today	Introduction, anti-RE, functionality	F0, F1, F2
13	Binary structure, crypto, processors, PE format	F3, F4, F5, F6, F7, F8
14	Hiding, reporting, obfuscation, behaviour	F9, F10, F11, F12

© Billy Forrest

SESSION OBJECTIVES

By the end of this session you will be able to

- **Explain the goals of malware analysis** determining if a file is malicious and what it can do
- **Define indicators of compromise** the IOC types and the signatures built from them
- **Share IOCs safely** using STIX, TAXII, CybOX and MAEC
- **Apply OPSEC** the do's and don'ts that keep analysis safe
- **Describe anti-reverse engineering** anti-disassembly, anti-debugging, anti-virtualisation, anti-sandboxing
- **Explain unpacking** static, automatic dynamic and manual, and the role of UPX
- **Identify malware functionality** using static analysis, strings, import hashing and PEiD

© Billy Forrest

The vocabulary of malware analysis

Term	Meaning
APT	Advanced persistent threat — a sophisticated, long-running campaign
C2	Command and control — the channel malware uses to receive instructions
IDS	Intrusion detection system
IOC	Indicator of compromise
PE	Portable executable — the Windows executable format
VM / Vbox	Virtual machine / VirtualBox — a sandbox for safe analysis

© Billy Forrest

Goals of malware analysis

- **Determine if a file is malicious** the first and most important judgement
- **Extract indicators of compromise** the artefacts that allow detection elsewhere
- **Create host-based signatures** for AV and host intrusion detection
- **Create network signatures** to spot C2 and exfiltration traffic
- **Determine the capabilities** what the malicious code can actually do
- **Update existing signatures** when a sample exposes new behaviour

© Billy Forrest

What counts as an IOC?

- **File hashes** cryptographic digests such as MD5, and fuzzy hashes such as SSDeep
- **Filenames and folder locations** the paths the malware creates or persists in
- **Registry keys** persistence and configuration locations
- **Strings** URLs, domains, IP addresses, commands
- **Network traffic patterns** the shape of C2 communication
- **Binary functions and encryption** and developer toolmarks

© Billy Forrest

Signatures built from IOCs

Signature type	Purpose
Anti-virus signatures	Host-based detection of known files
IDS signatures	Snort and Suricata rules for network detection
File / fuzzy hash databases	Rapid matching of known-good and known-bad files
YARA signatures	Flexible pattern and rule matching
HIDS signatures	Host intrusion detection
SIEM signatures	Correlation across security event logs

© Billy Forrest

Sharing indicators: STIX, TAXII, CybOX & MAEC

Format	What it does
STIX	Structured Threat Information eXpression — a language for sharing threat intelligence
TAXII	Trusted Automated eXchange of Indicator Information — the transport for sharing it
CybOX	Cyber Observable eXpression — describing the observable artefacts
MAEC	Malware Attribute Enumeration and Characterisation — describing malware attributes

© Billy Forrest

Common IOC file formats

- **Snort and Suricata IDS signatures** network rule formats
- **YARA signatures** pattern matching rules
- **OpenIOC signatures** the Mandiant open format
- **CSV** simple structured tabular sharing
- **Stuxnet** a landmark case — four zero-days, industrial PLC targeting, discovered June 2010

© Billy Forrest

Operations security — the don'ts

- **Do not allow malware to connect back** to the attacker from your lab
- **Do not manually connect** to attacker infrastructure
- **Do not scan attacker infrastructure** no port or vulnerability scans
- **Do not use discovered credentials** such as FTP usernames or passwords

© Billy Forrest

Operations security — the do's

- **Neuter extensions** rename bad.exe to bad.exe_ so it cannot be run accidentally
- **Password-protect archives** zip samples with the password 'infected'
- **Clearly label media** any portable media containing malicious code
- **Use a throwaway VM** never your corporate gold build image
- **Disable USB autorun** and beware browsers preloading links you search

© Billy Forrest

OPSEC: staying anonymous

- **Sometimes you must reach attacker infrastructure** for example to download exploit-kit payloads
- **The Onion Router (TOR)** routes traffic through an anonymous overlay network
- **VPN services** mask your true source address
- **Cloud upload risk** files sent to VirusTotal are shared with subscribers and AV vendors

© Billy Forrest

Identifying what a sample does — statically

- **Static analysis** examining code, binaries and headers without executing — safe and efficient
- **Information extraction** embedded strings, imports, exports, file headers
- **Strings utility** extracts printable ASCII and Unicode sequences — URLs, paths, commands
- **Import hashing (imphash)** a Mandiant technique hashing libraries, API names and their order
- **NSRL** National Software Reference Library — known-good hash sets to filter out
- **PEiD** detects common packers, cryptors and compilers

© Billy Forrest

DISCUSSION

Why is OPSEC failure so damaging in a malware investigation?

Think about what happens if you let a sample phone home, or you upload a unique sample to a public scanning service while the actor is still active.

OPSEC protects both the investigation and the wider defence — a single leak can tip off the adversary and end the campaign.

© Billy Forrest

The aim: conceal, but stay recoverable

- **Concealment goals** most packers try to conceal information — intellectual property or malware
- **Recovery requirement** the information must be recoverable at run time
- **The unpacking code does the work** the packer ships a stub that restores the original code
- **Anti-RE categories** anti-disassembly, anti-debugging, anti-virtualisation, anti-sandboxing

© Billy Forrest

Anti-disassembly methods

- **Overlapping instructions** jumping into the middle of another instruction
- **Linear disassembly abuse** garbage following a call, jump or conditional jump
- **Stack abuse** modifying return addresses, e.g. call $+5; add [esp],5; retn
- **Purpose** inhibits static analysis — disassembly and decompilation

© Billy Forrest

Anti-debugging and anti-virtualisation

- **Anti-debugging** inhibits dynamic analysis — debugging and tracing
- **Anti-virtualisation** detects the VM and changes behaviour or refuses to run
- **Anti-sandboxing** detects the analysis environment and evades it
- **High-profile bugs** known flaws in common debuggers and disassemblers are weaponised
- **Defeat** identify and neutralise the checks, then unpack and analyse

© Billy Forrest

Unpacking the binary

1. **Load the packed binary in a debugger**
   start execution in a controlled way
2. **Identify the unpacking code**
   and any anti-debugging code
3. **Identify the original entry point (OEP)**
   the start of the real program
4. **Run to the OEP**
   let the unpacking stub do its work
5. **Dump the process**
   capture the unpacked image from memory
6. **Reconstruct the import table**
   restore the functions called from other libraries

© Billy Forrest

Static, dynamic and manual unpacking

Approach	How it works	Examples
Static unpacking	Reconstructs the original from the packed binary; packed code does not run	UPX (single), PE Explorer (multiple)
Automatic dynamic	Uses the unpacking stub so packed code potentially runs	UnASPack (single), FUU (multiple)
Manual	Debug, dump and rebuild imports by hand	Any packer, most effort

© Billy Forrest

Group F at a glance

Ref	Skill area	Core idea
F0	Introduction to Malware Analysis	Goals, IOCs, sharing, OPSEC
F1	Windows Anti-Reverse Engineering	Anti-disassembly, anti-debugging, anti-virtualisation, anti-sandboxing, unpacking
F2	Functionality Identification	Static analysis, strings, import hashing, PEiD

© Billy Forrest

KEY TAKEAWAYS

- **Malware analysis has goals** judge maliciousness, extract IOCs, build signatures, assess capability
- **IOCs drive detection** hashes, filenames, registry keys, strings, network patterns, signatures
- **Sharing uses standards** STIX, TAXII, CybOX and MAEC
- **OPSEC protects the work** neuter files, protect archives, never let samples connect out
- **Anti-RE is an arms race** anti-disassembly, anti-debugging, anti-virtualisation, anti-sandboxing — beaten by unpacking
- **Functionality is identified** statically, with strings, imports, hashing and PEiD

© Billy Forrest

QUIZ

confirmation quiz (Group F I)

- Which sharing standard transports threat intelligence between parties?
- Why must you never allow malware to connect back to its C2?
- What does the Strings utility extract from a binary?
- What is the original entry point (OEP)?
- Which anti-RE technique inhibits static analysis?

Answers in the confirmation quiz — aim for 60 percent before leaving.

© Billy Forrest

NEXT SESSION

Next session: Binary Structure, Crypto, Processors & PE

- Windows NT architecture: kernel vs user mode, rings, processes
- Windows API development and how malware calls the OS
- Binary code structure and calling conventions
- Cryptographic techniques and processor architectures (x86/x64 registers)
- Windows executable file formats — the PE format in detail

Reading: Knowledge Group F source material, F3–F8.

© Billy Forrest



MY CYBER ACADEMY

Unit U13 of 14

CREST Practitioner Intrusion Analyst (CPIA)

Reverse Engineering Malware II: Binary Structure, Crypto, Processors & PE

Windows internals, the API, calling conventions, cryptography, the x86/x64 processor and the PE format.

GROUP F | PE FORMAT | PROCESSORS

© Billy Forrest

Unit U13 covers six skill areas

Ref	Skill area	Focus
F3	Windows NT Architecture	Kernel vs user mode, processes
F4	Windows API Development	How code calls the OS
F5	Binary Code Structure	Calling conventions, compiler constructs
F6	Cryptographic Techniques	Encryption and encoding in malware
F7	Processor Architectures	x86/x64 registers, virtual memory
F8	Windows Executable File Formats	The PE format in detail

© Billy Forrest

SESSION OBJECTIVES

By the end of this session you will be able to

- **Describe Windows NT architecture** kernel and user mode, privilege rings, processes
- **Explain the Windows API** and how malware calls OS functions at load and run time
- **Describe binary code structure** calling conventions, stack frames and virtual function tables
- **Recognise cryptographic techniques** the encryption and encoding used in malware
- **Describe the x86/x64 processor** registers, instruction set and virtual memory
- **Read the PE format** headers, sections, imports, exports, resources and relocations

© Billy Forrest

Kernel mode and user mode

- **Two privilege levels** the processor enforces rings that control access to resources
- **Kernel mode (Ring 0)** highest privilege — drivers, the core of the OS, direct hardware access
- **User mode (Ring 3)** applications run here with restricted access to hardware
- **Context switching** transitions between modes are mediated by the OS
- **Malware targeting** kernel-mode rootkits are far harder to detect than user-mode code

© Billy Forrest

Processes, threads and the executive

Component	Role
Process	An isolated address space; each process gets its own memory segment
Thread	The unit of execution within a process
Executive	The kernel-mode services — memory, I/O, process and object management
Subsystems	Environment subsystems present the Win32 API to applications
Registry	Configuration store malware abuses for persistence

© Billy Forrest

How code calls the operating system

- **The Win32 API** the set of functions applications call to use Windows services
- **Imports at load time** functions resolved via the import table in the PE
- **Imports at run time** LoadLibrary / GetModuleHandle plus GetProcAddress
- **Malware preference** run-time resolution avoids obvious import strings
- **API hashing** malware stores a hash of the function name and resolves it dynamically

© Billy Forrest

API hashing in practice

- **Avoids readable strings** no plain-text API names for an analyst to spot
- **Less data to store** a single 32-bit hash replaces a long name
- **Bespoke resolvers** custom GetModuleHandle and GetProcAddress replacements
- **Typical hash** ROL(hash, 5) XOR each byte of the name

© Billy Forrest

Calling conventions

Convention	Argument passing	Stack cleanup
cdecl	Arguments pushed on the stack, right to left	Caller cleans up
stdcall	Arguments pushed on the stack, right to left	Callee cleans up
fastcall	First arguments passed in registers (ECX, EDX)	Callee cleans up

© Billy Forrest

Compiler-generated constructs

- **Function prologue and epilogue** setup and teardown of the stack frame
- **Stack frames** local variables, saved registers and return addresses live here
- **C++ objects** methods compiled as functions; objects carry a pointer to their data
- **Virtual function table (vtable)** a table of pointers to the object's virtual methods
- **Recognising these** lets you reconstruct high-level intent from raw assembly

© Billy Forrest

Crypto in malware

- **Hashing** fingerprinting, integrity checks, API and string obfuscation
- **Symmetric encryption** stream and block ciphers to hide payloads and C2 traffic
- **Asymmetric encryption** public-key crypto for key exchange and secure C2
- **Encoding vs encryption** Base64 is encoding, not encryption — it offers no real secrecy
- **Custom algorithms** malware authors often invent bespoke ciphers to defeat signatures

© Billy Forrest

The x86 instruction set and registers

- **Instruction set** the interface between CPU hardware and the software running on it
- **An instruction** a binary message to the CPU to perform an action next cycle
- **OpCode and operands** an operation code plus addresses, representable by mnemonics
- **RAX** 64-bit general register; splits into EAX (32), AX (16), AH and AL (8-bit halves)
- **RBX / RCX / RDX** RCX is the loop counter; RDX is the overflow for multiply and divide

© Billy Forrest

Stack and string registers

Register	Purpose
RBP	Base pointer — stack operations and location counters
RSP	Stack pointer — top of the stack
RSI	Source index — source for string commands, increments when used
RDI	Destination index — destination for string commands, increments when used

© Billy Forrest

x64 and virtual memory

- **x64 registers** identical to x86 plus R8 to R15 — 16 general-purpose registers total
- **Physical memory** any storage device — cache, RAM, disk, SSD — each with a physical address
- **Virtual memory** each process gets its own segment, possibly on different physical modules, appearing continuous
- **The MMU** Memory Management Unit translates virtual addresses into physical ones

© Billy Forrest

Virtualisation and its flaws

- **Virtualisation** hardware virtualisation of whole machines (VMs)
- **Containerisation** operating-system-level virtualisation
- **Privilege rings** the host OS lives at ring 0; the guest runs at a higher ring
- **Meltdown** a flaw letting a malicious process access all of RAM
- **Spectre** a flaw allowing access to protected data by observing processor actions

© Billy Forrest

The Portable Executable (PE) format

- **Common PE types** EXE, SCR, DLL, CPL, OCX, SYS
- **Three data parts** a single header, multiple sections, and multiple data directories
- **Data directories** may be stored within or after the sections
- **PE types** EXE is an executable program; DLL is a dynamically-linkable library
- **Reading a PE** is the first step in identifying malicious structure

© Billy Forrest

DOS, NT, file and optional headers

Header	Key contents
IMAGE_DOS_HEADER	Starts with 'MZ'; 16-bit DOS stub; pointer to the NT headers
IMAGE_NT_HEADERS	Starts with 'PE' plus two nulls; contains the file and optional headers
IMAGE_FILE_HEADER	CPU required, number of sections, timestamp, symbol table, EXE/DLL flag
IMAGE_OPTIONAL_HEADER	Versions, base addresses, size of image, entry point, data directories

© Billy Forrest

PE sections

- **Code** executable content; every EXE has at least one code section
- **Data** information used by the code — read-only, read-write initialised or uninitialised
- **Imports** functions resolved from other libraries at load or run time
- **Exports** functions callable by other executables — mostly in DLLs
- **Resources** dialog boxes, icons, images, strings, manifests
- **Relocations** the changes needed when a file loads away from its preferred base

© Billy Forrest

Imports by hash and ASLR

- **Imports by hash** malware hashes API names to avoid readable strings
- **Run-time resolving** missing imports indicate LoadLibrary / GetProcAddress
- **Relocations** most PEs must be relocatable
- **ASLR** address space layout randomisation loads at a random base, triggering relocations

© Billy Forrest

How a PE loads, and spotting bad ones

- **The PE loader** parse headers, parse section table, map sections, resolve imports, call entry point
- **Tools** DumpBin (command line), PE viewer GUI, hex editor
- **Indicators of a bad PE** multiple or bogus headers, missing or extra sections, bogus section data
- **Inspection** check timestamps, versions, data-directory sizes, section names and permissions

© Billy Forrest

CONSOLIDATION · GROUP F

F3 to F8 at a glance

Ref	Skill area	Core idea
F3	Windows NT Architecture	Kernel vs user mode, processes and threads
F4	Windows API Development	Load-time and run-time imports, API hashing
F5	Binary Code Structure	Calling conventions, stack frames, vtables
F6	Cryptographic Techniques	Hashing, symmetric, asymmetric, encoding
F7	Processor Architectures	x86/x64 registers, virtual memory, MMU
F8	Executable File Formats	PE headers, sections, imports, exports, relocations

© Billy Forrest

KEY TAKEAWAYS

- **Windows has two modes** kernel (ring 0) and user (ring 3), mediated by the OS
- **The API is the surface** malware imports at load or run time, often resolving by hash
- **Calling conventions matter** cdecl, stdcall and fastcall define who cleans the stack
- **Crypto hides and verifies** hashing, symmetric and asymmetric ciphers, and encoding
- **The processor exposes registers** x86/x64 registers, virtual memory and the MMU
- **The PE format is readable** headers, sections, imports, exports, resources and relocations

© Billy Forrest

QUIZ

confirmation quiz (Group F II)

- Which PE header starts with the 'MZ' signature?
- What does the MMU translate?
- In the stdcall convention, who cleans up the stack?
- Which register is used as the loop counter?
- What does ASLR do that requires relocations?

Answers in the confirmation quiz — aim for 60 percent before leaving.

© Billy Forrest

NEXT SESSION

Next session: Hiding, Reporting, Obfuscation & Behaviour

- Hiding techniques: process injection, rootkits, SSDT patching, filter drivers
- Malware reporting: YARA, SSDeep, infection vectors, imphash
- Binary obfuscation and behavioural analysis
- A full review of all six knowledge groups (A–F) to tie the course together

Reading: Knowledge Group F source material, F9–F12.

© Billy Forrest



MY CYBER ACADEMY

Unit U14 of 14

CREST Practitioner Intrusion Analyst (CPIA)

Reverse Engineering Malware III & Exam Consolidation

Hiding, reporting, obfuscation and behaviour — then a full review of all six knowledge groups and exam preparation.

GROUP F | MALWARE REPORTING | EXAM CONSOLIDATION

© Billy Forrest

THE FINAL SESSION

Completing Group F, then the whole course

- **Four remaining skill areas** F9 hiding, F10 reporting, F11 obfuscation, F12 behaviour
- **Then the consolidation** a review of all six knowledge groups, A to F
- **Exam preparation** strategy, question technique and a full-length quiz
- **You have covered** the complete CREST CPIA syllabus across 14 sessions

© Billy Forrest

What is a rootkit?

- **Administrative privilege** rootkits obtain administrative privileges on the system
- **Origins** originally used by network and system administrators
- **Now stealth malware** widely referred to as stealth-type malware
- **Capabilities** maintain presence, execute remote commands, disable security software, alter files, install more malware

© Billy Forrest

How rootkits remain hidden

- **Clear security logs** erase user audit logs to hide activity
- **Delete generated evidence** and hide files and directories
- **Timing attacks** sleep to avoid automated malware analysis
- **Process injection** such as DLL injection into svchost.exe or explorer.exe
- **SSDT patching** redirect kernel-mode function pointers
- **Filter drivers and process list manipulation** to hide their own presence

© Billy Forrest

Process injection techniques

Technique	Approach
Classic DLL injection	Find target, allocate space, load code into the target, execute it remotely
PE injection	Find target, allocate space, copy the malicious code in, execute it remotely
Process hollowing	Find target, unmap its legitimate code, overwrite with malicious code, execute it remotely

© Billy Forrest

Kernel hiding mechanisms

- **SSDT** the System Service Descriptor Table holds pointers to kernel-mode functions
- **SSDT fields** ServiceTable, CounterTable, ServiceLimit, ArgumentTable
- **SSDT patching** redirecting those pointers so the system lies about what exists
- **Filter drivers** sit in the driver stack and filter requests, hiding files or resources
- **Process list manipulation** image-name and parent-process exploitation, and process hollowing

© Billy Forrest

Signatures and fuzzy hashing

- **SSDeep** fuzzy hashing that produces comparable hashes for similar files
- **Signature identification** identifying a sample so it can be reported and detected
- **YARA** open-source, multi-platform pattern, string and signature matching
- **YARA rules** logical rules using Perl-based regular expressions
- **Footprint and cleanup** recording the sample's footprint to support removal

© Billy Forrest

Writing effective YARA rules

1. **Identify the target**
   select the executable to analyse
2. **Extract packed strings**
   run string extraction in ASCII and Unicode modes
3. **Extract unpacked strings**
   execute the malware and gather more strings with Process Explorer
4. **Develop rules**
   create separate rules for the packed and unpacked states
5. **Match on unique strings**
   malware-specific and incident-specific indicators

© Billy Forrest

How malware gets in

- **Infected executables and documents** malware in EXEs, DLLs, Office macros and JavaScript
- **Drive-by downloads** unintentional downloads from compromised websites
- **USB and shared drives** infected files copied or auto-executed from removable media
- **Passive exploitation** unpatched services exploited without user interaction
- **Email-based attacks** phishing with malicious attachments or links

© Billy Forrest

Reporting supports detection

- **Static analysis** examining a file without executing it — safe and efficient
- **Import hashing (imphash)** hashes libraries, API names and their order to group related samples
- **NSRL** the National Software Reference Library filters out known-good files
- **PEiD** detects packers, cryptors and compilers
- **Strings utility** extracts printable ASCII and Unicode sequences for clues

© Billy Forrest

Obfuscating the binary

- **Packing and compression** the original code is hidden until run time
- **Encryption** payload and strings encrypted so static analysis reveals little
- **String obfuscation** encoding strings to defeat signature and strings-utility detection
- **Anti-RE integration** obfuscation combined with anti-disassembly and anti-debugging
- **The goal** raise the cost of analysis and delay detection

© Billy Forrest

Watching the malware run

- **Dynamic analysis** run the sample in a controlled environment and observe
- **Sandbox or VM** isolate the sample so it cannot harm the host
- **Observe behaviour** files created, registry keys written, processes spawned, network traffic
- **Timing awareness** some malware sleeps or detects the sandbox to avoid revealing behaviour
- **Combine with static** behavioural findings complement static analysis of the binary

© Billy Forrest

DISCUSSION

Why is behavioural analysis alone insufficient for a full malware assessment?

Think about timing attacks, anti-sandboxing, and the fact that a sample may only reveal part of its behaviour in any single run.

Robust analysis combines static inspection, unpacking and behavioural observation.

© Billy Forrest

COURSE CONSOLIDATION

The six knowledge groups

Group	Knowledge area	Sessions
A	Soft Skills & Incident Handling	1
B	Core Technical Skills	2–4
C	Background Info Gathering & OSINT	5
D	Network Intrusion Analysis	6–8
E	Analysing Host Intrusions	9–11
F	Reverse Engineering Malware	12–14

© Billy Forrest

Soft skills, incident handling and core technical skills

- **Group A** the analyst role, bias, IR lifecycle, kill chain, law, TAME, risk treatment
- **Group B** IP protocols, the OSI model, HTTP, DNS, SMB, tools, fingerprinting
- **Group B continued** cryptography, permissions and host analysis
- **Exam focus** scenario-based application of the IR lifecycle and network fundamentals

© Billy Forrest

COURSE CONSOLIDATION · GROUPS C & D

OSINT and network intrusion analysis

- **Group C** background information gathering and open-source investigation
- **Group D** packet capture, logs, network configuration
- **Group D continued** C2 detection, exfiltration, incoming attacks, internal spread, false positives
- **Exam focus** reading captures and logs, identifying intrusion patterns

© Billy Forrest

COURSE CONSOLIDATION · GROUPS E & F

Host intrusions and reverse engineering

- **Group E** acquisition, Windows file system, registry, memory, infection, rootkits
- **Group F** malware analysis, anti-RE, PE format, obfuscation, behaviour
- **Group F continued** hiding techniques, reporting, unpacking, functionality
- **Exam focus** connecting host artefacts to malware behaviour

© Billy Forrest

KEY TAKEAWAYS

- **Rootkits hide at kernel or user level** SSDT patching, filter drivers, process injection, process list manipulation
- **Reporting uses signatures** SSDeep fuzzy hashing, YARA rules, imphash and NSRL
- **Obfuscation raises the cost** packing, encryption and string obfuscation delay analysis
- **Behaviour complements static** run the sample in a sandbox and observe files, registry and network
- **The syllabus is complete** all six knowledge groups — A through F — are now covered

© Billy Forrest

QUIZ

final confirmation quiz (all groups)

- Which SSDT fields hold the kernel function pointers?
- What does process hollowing unmap before writing malicious code?
- Why is Base64 NOT considered encryption?
- Which YARA feature lets rules match a threshold of strings?
- Name the IR phase that follows analysis.

Answers in the confirmation quiz — aim for 60 percent before the exam.

© Billy Forrest

EXAM PREPARATION

Your route to the CPIA exam

- **Revisit every confirmation quiz** retake until 60 percent without guessing
- **Practise scenario reasoning** the exam is scenario-based — apply, don't just recall
- **Revise the six groups** A Soft Skills, B Core Technical, C OSINT, D Network, E Host, F Reverse Engineering
- **Know your tools** Strings, PEiD, DumpBin, YARA, SSDeep, imphash, the debugger
- **Plan the 120 minutes** time per question, flag and return, read every explanation
- **Good luck — you are ready** fourteen sessions, six knowledge groups, one certification

© Billy Forrest